AIFP SDK Reference

Document: AIFP-DOC-11 · **Version:** 1.0.0 · **Governed by:** AIFP-1 (Doc 01)

Aligned to: OpenAPI 3.1 (Doc 08) · JSON Schemas (Doc 10) · Agent SDK Spec (Doc 03)

> Every SDK is a thin, idiomatic wrapper over the same protocol surface. Method names,
>
> arguments, and return shapes map 1:1 to the OpenAPI operations. Where an SDK and the
>
> protocol disagree, **AIFP-1 governs**. All SDKs implement the same **4-step loop**:
>
> `402 → quote → pay → retry-with-receipt`, plus stateless receipt verification.

Common model (all languages)

Concept	Shape
`Merchant` / `Verifier`	Verifies receipts statelessly (JWKS cache).
`Quote`	`{ quote_id, amount, currency, accepted_assets, accepted_chains, nonce, expires_at }`
`Receipt`	`{ receipt_id, receipt(jwt), status, tx_ref, amount, fee, expires_at }`
`Passport`	`{ passport_id, agent_id, reputation(0–1000), risk(0–100), trust_level }`
Errors	Typed exceptions mapping the `AIFP-*` registry (Doc 01 App. C).

Shared options: `apiKey` (`sk_live_*` / `sk_test_*`), `baseUrl`

(`https://api.aifinpay.io` / sandbox), `timeoutMs` , `maxRetries` (default 5, exp backoff

+ jitter, base 200ms, cap 30s). **Idempotency-Key** is auto-generated for `pay()` unless

supplied.

Pricing model

Tier	Starts From	Typical Action
`complex`	`$0.00006`	Search, aggregation, multi-source queries, higher compute
`premium`	`$0.00010`	AI inference, GPU workloads, deep analytics, premium data

AiFinPay applies a **1% protocol fee** to every successful transaction. The remaining **99%** is settled to the merchant, excluding applicable payment network or settlement costs.

1. TypeScript / Node — `@aifinpay/agent`, `@aifinpay/merchant`

Install: `npm i @aifinpay/agent @aifinpay/merchant`

Classes

- `Agent` — autonomous client.
- `Wallet` — provisioning/funding/budgets.
- `MerchantVerifier` — stateless receipt verification.

Agent

```
import { Agent } from "@aifinpay/agent"

const agent = new Agent({ apiKey: process.env.AIFP_KEY!, walletId: "wlt_3a1b" });

// One-shot pay-through (handles 402 -> quote -> pay -> retry)
const res = await agent.get("https://merchant.example.com/api/data");
console.log(res.data, res.aifp.receiptId);

// Explicit steps
const quote   = await agent.quote({ merchantId: "mrch_9f3a1c2b", resource: "/api/data", pricingTier:
"standard" });
const receipt = await agent.pay({ quoteId: quote.quoteId, walletId: "wlt_3a1b", asset: "USDC", chain:
"polygon" });

// Budget policy (throws BudgetExceededError -> AIFP-403-BUDGET-EXCEEDED)
await agent.setBudget({ window: "day", capUsd: "50.00" });
```

Interfaces

```
interface QuoteRequest { merchantId: string; resource: string; pricingTier?:
"standard"|"complex"|"premium"; currency?: "USD"; }
interface PayRequest   { quoteId: string; walletId: string; asset: "USDC"|"USDT"|"PYUSD"; chain: Chain;
idempotencyKey?: string; }
interface Receipt      { receiptId: string; receipt: string; status:
"settled"|"settling"|"expired"|"revoked"; txRef?: string; amount: string; fee?: string; expiresAt: string;
}
```

MerchantVerifier

```
import { MerchantVerifier } from "@aifinpay/merchant";
const verifier = new MerchantVerifier({ merchantId: "mrch_9f3a1c2b" }); // caches JWKS
const { valid, claims, reason } = await verifier.verify(jwt, { resource: "/api/data" });
```

Events / Callbacks

```
agent.on("quote", q => {});
agent.on("paid", r => {});
```

```
agent.on("retry", ({ attempt, code }) => {});
verifier.on("rejected", ({ code }) => {}); // e.g. AIFP-422-SIGNATURE
```

Best practices

Reuse one `MerchantVerifier` (persistent JWKS + nonce store). Never re-fetch JWKS per request. Always set a budget on production agents.

---

2. Python — `aifinpay-agent`, `aifinpay-merchant`

Install: `pip install aifinpay-agent aifinpay-merchant`

```
from aifinpay_agent import Agent, BudgetExceeded

agent = Agent(api_key="sk_live_...", wallet_id="wlt_3a1b")

# Pay-through
resp = agent.get("https://merchant.example.com/api/data")
print(resp.json(), resp.aifp.receipt_id)

# Explicit
quote = agent.quote(merchant_id="mrch_9f3a1c2b", resource="/api/data", pricing_tier="standard")
receipt = agent.pay(quote_id=quote.quote_id, wallet_id="wlt_3a1b", asset="USDC", chain="polygon")

agent.set_budget(window="day", cap_usd="50.00")  # raises BudgetExceeded later
```

```
# Async variant
from aifinpay_agent.aio import AsyncAgent
async with AsyncAgent(api_key="sk_live_...") as a:
    r = await a.get(url)
```

```
# Merchant verification (stateless)
from aifinpay_merchant import verify_receipt
ok, claims = verify_receipt(jwt, merchant_id="mrch_9f3a1c2b", resource="/api/data")
```

Classes: `Agent`, `AsyncAgent`, `Wallet`, `Passport`. **Callbacks:** `on_quote`, `on_paid`, `on_retry`. **Exceptions:** `PaymentRequired`, `BudgetExceeded`, `InvalidReceipt`, `QuoteExpired`, `RateLimited` (map to the `AIFP-*` codes).

---

3. Go — `github.com/aifinpay/aifp-go`

Install: `go get github.com/aifinpay/aifp-go`

```
import "github.com/aifinpay/aifp-go"

ag := aifp.NewAgent(aifp.Config{APIKey: os.Getenv("AIFP_KEY"), WalletID: "wlt_3a1b"})
```

```
// Pay-through
res, err := ag.Get(ctx, "https://merchant.example.com/api/data")

// Explici
q, _ := ag.Quote(ctx, aifp.QuoteRequest{MerchantID: "mrch_9f3a1c2b", Resource: "/api/data", PricingTier:
aifp.Standard})
r, _ := ag.Pay(ctx, aifp.PayRequest{QuoteID: q.QuoteID, WalletID: "wlt_3a1b", Asset: aifp.USDC, Chain:
aifp.Polygon})

// Merchant verify (no network call)
v := aifp.NewVerifier("mrch_9f3a1c2b")
claims, err := v.Verify(jwt, aifp.VerifyOpts{Resource: "/api/data"})
```

Interfaces: `Agent`, `Verifier`, `Wallet`. Errors implement `aifp.Error` exposing `.Code()` (e.g. `aifp.ErrBudgetExceeded`). Hooks via `Config.OnRetry func(attempt int, code string)`.

---

4. Rust — `aifinpay`

Install: `cargo add aifinpay`

```
use aifinpay::{Agent, QuoteRequest, PayRequest, Asset, Chain, PricingTier};

let agent = Agent::new("sk_live_...").wallet("wlt_3a1b");
let res = agent.get("https://merchant.example.com/api/data").await?;

let q = agent.quote(QuoteRequest{ merchant_id:"mrch_9f3a1c2b".into(), resource:"/api/data".into(),
pricing_tier:PricingTier::Standard, ..Default::default() }).await?;
let r = agent.pay(PayRequest{ quote_id:q.quote_id, wallet_id:"wlt_3a1b".into(), asset:Asset::Usdc,
chain:Chain::Polygon }).await?;

// Verify
let v = aifinpay::Verifier::new("mrch_9f3a1c2b");
let claims = v.verify(&jwt, "/api/data")?; // Result<Claims, AifpError>
```

`AifpError` is an enum mirroring the registry (`AifpError::BudgetExceeded`, `::Signature`, `::QuoteExpired`, …). Async on Tokio; `verify` is sync and allocation-light.

---

5. Java — `io.aifinpay:aifp`

Install: `implementation "io.aifinpay:aifp:1.0.0"`

```
Agent agent = Agent.builder().apiKey(System.getenv("AIFP_KEY")).walletId("wlt_3a1b").build();
PayThroughResponse res = agent.get("https://merchant.example.com/api/data");

Quote q = agent.quote(QuoteRequest.builder()
    .merchantId("mrch_9f3a1c2b").resource("/api/data").pricingTier(PricingTier.STANDARD).build());
Receipt r = agent.pay(PayRequest.builder()
    .quoteId(q.getQuoteId()).walletId("wlt_3a1b").asset(Asset.USDC).chain(Chain.POLYGON).build());
```

```
MerchantVerifier v = new MerchantVerifier("mrch_9f3a1c2b");
VerifyResult vr = v.verify(jwt, "/api/data");
```

Checked exception `AifpException` exposes `getCode()`. Listeners via `agent.addListener(AifpListener)`. Thread-safe verifier with internal JWKS cache.

---

6. PHP — `aifinpay/aifp`

Install: `composer require aifinpay/aifp`

```
use AiFinPay\Agent; use AiFinPay\MerchantVerifier;

$agent = new Agent(['apiKey' => getenv('AIFP_KEY'), 'walletId' => 'wlt_3a1b']);
$res = $agent->get('https://merchant.example.com/api/data');

$quote   = $agent->quote(['merchantId' => 'mrch_9f3a1c2b', 'resource' => '/api/data', 'pricingTier' =>
'standard']);
$receipt = $agent->pay(['quoteId' => $quote->quoteId, 'walletId' => 'wlt_3a1b', 'asset' => 'USDC', 'chain'
=> 'polygon']);

$verifier = new MerchantVerifier('mrch_9f3a1c2b');
[$ok, $claims] = $verifier->verify($jwt, '/api/data');
```

Throws `AiFinPay\Exception\AifpException` with `->getCode()`. PSR-18 HTTP client + PSR-3 logging supported.

---

7. C# / .NET — `AiFinPay`

Install: `dotnet add package AiFinPay`

```
var agent = new Agent(new AgentOptions { ApiKey = Env("AIFP_KEY"), WalletId = "wlt_3a1b" });
var res = await agent.GetAsync("https://merchant.example.com/api/data");

var quote   = await agent.QuoteAsync(new QuoteRequest { MerchantId = "mrch_9f3a1c2b", Resource =
"/api/data", PricingTier = PricingTier.Standard });
var receipt = await agent.PayAsync(new PayRequest { QuoteId = quote.QuoteId, WalletId = "wlt_3a1b", Asset =
Asset.USDC, Chain = Chain.Polygon });

var verifier = new MerchantVerifier("mrch_9f3a1c2b");
var (valid, claims) = await verifier.VerifyAsync(jwt, resource: "/api/data");
```

`AifpException.Code` carries the registry code. Events: `agent.OnQuote`, `agent.OnPaid`, `agent.OnRetry`. Verifier is `IDisposable` and caches JWKS.

Cross-language guarantees

1. **Identical pricing & enums** — action pricing tiers (Standard from $0.00001, Complex from $0.00006, Premium from $0.00010), assets (USDC/USDT/PYUSD), and the 12-chain set are the same everywhere.
2. **Protocol fee** — every SDK surfaces the 1% AiFinPay protocol fee and merchant net settlement amount where available.
3. **Stateless verify** — every `Verifier` performs the AIFP-1 §7.4 10-step check with no network call; only JWKS is cached.
4. **Idempotency** — `pay()` auto-attaches an `Idempotency-Key` (24h window) unless overridden.
5. **Retries** — exponential backoff with jitter (base 200ms, cap 30s, 5 attempts); `402` / `409` are never blindly retried.
6. **Errors** — every SDK exposes the canonical `AIFP-*` code; new codes (e.g. `AIFP-403-BUDGET-EXCEEDED`) surface uniformly.